

Werewolf Game: LLM Reference Documentation

Document Purpose

This document provides structured, machine-readable information about the Werewolf social deduction game for Large Language Model parsing and implementation. All data is formatted for programmatic extraction and game logic implementation.

Game Metadata

```
{
  "game_name": "Werewolf",
  "alternative_names": ["Mafia", "City of Palermo"],
  "category": "social_deduction",
  "year_created": 1986,
  "creator": "Dimitry Davidoff",
  "origin": "Moscow State University, Psychology Department",
  "player_count": {
    "minimum": 7,
    "maximum": 75,
    "optimal": "8-16",
    "note": "Odd numbers preferred for tiebreaker scenarios"
  },
  "game_duration": {
    "traditional": "30-90 minutes",
    "one_night_variant": "10-15 minutes"
  },
  "primary_mechanics": [
    "hidden_roles",
    "player_elimination",
    "voting",
    "bluffing",
```

```
"deduction",
"social_interaction"
]
}
```

Core Game Structure

Teams

```
{
"teams": [
{
"team_id": "werewolf",
"name": "Werewolf Team",
"alignment": "evil",
"information_level": "informed_minority",
"typical_ratio": 0.33,
"win_condition": "Achieve numerical parity with villagers OR
eliminate all villagers",
"abilities": ["knows_teammates", "night_kill"]
},
{
"team_id": "village",
"name": "Village Team",
"alignment": "good",
"information_level": "uninformed_majority",
"typical_ratio": 0.67,
"win_condition": "Eliminate all werewolves",
"abilities": []
},
{
"team_id": "neutral",
"name": "Neutral Roles",
"alignment": "neutral",
"information_level": "varies",
"typical_ratio": 0.0,
"win_condition": "Role-specific alternative conditions",
"abilities": []
}
}
```

```
]
}
```

Game Phases

```
{
  "phases": [
    {
      "phase_id": "night",
      "name": "Night Phase",
      "order": 1,
      "visibility": "eyes_closed",
      "duration": "2-5 minutes",
      "actions_allowed": ["role_abilities", "werewolf_kill",
        "seer_investigation", "doctor_protection"],
      "communication": "restricted_by_role",
      "description": "Players close eyes. Roles with night abilities perform
        actions in specific order."
    },
    {
      "phase_id": "day",
      "name": "Day Phase",
      "order": 2,
      "visibility": "public",
      "duration": "5-10 minutes",
      "actions_allowed": ["discussion", "accusation", "voting", "elimination"],
      "communication": "all_players",
      "description": "Players discuss, debate, and vote to eliminate
        suspected werewolf."
    }
  ],
  "cycle": "Phases alternate until win condition met"
}
```

Role Database

Role Schema

```
interface Role {  
  role_id: string;  
  name: string;  
  team: "werewolf" | "village" | "neutral";  
  priority: "essential" | "high" | "medium" | "low";  
  implementation_complexity: "simple" | "moderate" | "complex";  
  night_action: boolean;  
  action_order: number | null;  
  action_description: string;  
  win_condition: string;  
  special_mechanics: string[];  
  reveals_on_death: boolean;  
  recommended_player_count: string;  
}
```

Essential Roles (Priority: Essential)

```
[  
  {  
    "role_id": "villager",  
    "name": "Villager",  
    "team": "village",  
    "priority": "essential",  
    "implementation_complexity": "simple",  
    "night_action": false,  
    "action_order": null,  
    "action_description": "No special abilities. Uses logic and observation  
during day phase.",  
    "win_condition": "Eliminate all werewolves",  
    "special_mechanics": [],  
    "reveals_on_death": true,  
    "recommended_player_count": "any",  
    "quantity_formula": "total_players - special_roles - werewolves"  
  },  
  {  
    "role_id": "werewolf",  
    "name": "Werewolf",  
    "team": "werewolf",
```

```
"priority": "essential",
"implementation_complexity": "simple",
"night_action": true,
"action_order": 1,
"action_description": "During night, werewolves see each other and
collectively choose one player to eliminate.",
"win_condition": "Achieve parity with villagers or eliminate all
villagers",
"special_mechanics": ["sees_teammates", "collective_kill",
"must_blend_during_day"],
"reveals_on_death": true,
"recommended_player_count": "any",
"quantity_formula": "Math.floor(total_players / 3) with minimum 1"
},
{
"role_id": "seer",
"name": "Seer",
"team": "village",
"priority": "essential",
"implementation_complexity": "simple",
"night_action": true,
"action_order": 3,
"action_description": "Each night, learns if one chosen player is
werewolf or villager. Returns boolean: is_werewolf.",
"win_condition": "Eliminate all werewolves",
"special_mechanics": ["investigation", "information_asymmetry",
"high_value_target"],
"reveals_on_death": true,
"recommended_player_count": "7+",
"quantity_formula": "1"
},
{
"role_id": "doctor",
"name": "Doctor",
"team": "village",
"priority": "essential",
"implementation_complexity": "simple",
"night_action": true,
"action_order": 2,
```

```

"action_description": "Each night, protects one player from werewolf
elimination. Can protect self. If target matches werewolf kill, save is
announced.",
"win_condition": "Eliminate all werewolves",
"special_mechanics": ["protection", "can_self_target",
"save_revealed"],
"reveals_on_death": true,
"recommended_player_count": "8+",
"quantity_formula": "1"
}
]

```

High Priority Roles

```

[
{
"role_id": "hunter",
"name": "Hunter",
"team": "village",
"priority": "high",
"implementation_complexity": "moderate",
"night_action": false,
"action_order": null,
"action_description": "When eliminated (day or night), immediately
chooses one other player to eliminate.",
"win_condition": "Eliminate all werewolves",
"special_mechanics": ["revenge_kill", "death_trigger"],
"reveals_on_death": true,
"recommended_player_count": "9+",
"quantity_formula": "1"
},
{
"role_id": "tanner",
"name": "Tanner",
"team": "neutral",
"priority": "high",
"implementation_complexity": "simple",
"night_action": false,
"action_order": null,
"action_description": "No abilities. Wins only if eliminated by village

```

```

vote during day phase.",
"win_condition": "Be eliminated by village vote (not werewolf kill)",
"special_mechanics": ["alternative_win_condition",
"suicide_strategy"],
"reveals_on_death": true,
"recommended_player_count": "8+",
"quantity_formula": "1"
}
]

```

Medium Priority Roles (One Night Variant)

```

[
{
"role_id": "robber",
"name": "Robber",
"team": "village",
"priority": "medium",
"implementation_complexity": "moderate",
"night_action": true,
"action_order": 5,
"action_description": "Swaps their role card with another player's
card, then looks at their new role. Becomes that role but doesn't
perform its night action.",
"win_condition": "Based on final role after swap",
"special_mechanics": ["role_swap", "identity_change",
"one_night_only"],
"reveals_on_death": true,
"recommended_player_count": "5+ (One Night variant)",
"quantity_formula": "1"
},
{
"role_id": "troublemaker",
"name": "Troublemaker",
"team": "village",
"priority": "medium",
"implementation_complexity": "moderate",
"night_action": true,
"action_order": 6,
"action_description": "Swaps two other players' cards without looking

```

```

at them. Those players become new roles unknowingly.",
"win_condition": "Village team wins",
"special_mechanics": ["role_swap_others", "creates_confusion",
"one_night_only"],
"reveals_on_death": true,
"recommended_player_count": "5+ (One Night variant)",
"quantity_formula": "1"
},
{
"role_id": "minion",
"name": "Minion",
"team": "werewolf",
"priority": "medium",
"implementation_complexity": "simple",
"night_action": true,
"action_order": 2,
"action_description": "Sees who werewolves are (they give signal), but
werewolves don't know minion. Appears as villager to Seer.",
"win_condition": "Werewolf team wins",
"special_mechanics": ["knows_werewolves",
"not_detected_as_werewolf", "asymmetric_information"],
"reveals_on_death": true,
"recommended_player_count": "8+",
"quantity_formula": "1"
},
{
"role_id": "mason",
"name": "Mason",
"team": "village",
"priority": "medium",
"implementation_complexity": "simple",
"night_action": true,
"action_order": 3,
"action_description": "Two Masons see each other during night.
Provides confirmed village members.",
"win_condition": "Eliminate all werewolves",
"special_mechanics": ["paired_role", "mutual_confirmation"],
"reveals_on_death": true,
"recommended_player_count": "10+",

```



```
"quantity_formula": "2 (must use both)"
}
]
```

Advanced Roles (Low Priority)

```
[
{
"role_id": "witch",
"name": "Witch",
"team": "village",
"priority": "low",
"implementation_complexity": "complex",
"night_action": true,
"action_order": 2,
"action_description": "Has one-time kill potion and one-time revive
potion. Can use during game at night.",
"win_condition": "Eliminate all werewolves",
"special_mechanics": ["limited_use_abilities", "kill_power",
"revive_power"],
"reveals_on_death": true,
"recommended_player_count": "12+",
"quantity_formula": "1"
},
{
"role_id": "cupid",
"name": "Cupid",
"team": "village",
"priority": "low",
"implementation_complexity": "complex",
"night_action": true,
"action_order": 0,
"action_description": "First night only: creates bond between two
players (can include self). Bonded players win/lose together
regardless of team.",
"win_condition": "Bonded partner survives",
"special_mechanics": ["creates_new_team", "first_night_only",
"changes_win_conditions"],
"reveals_on_death": true,
"recommended_player_count": "12+",
```

```
"quantity_formula": "1"
},
{
  "role_id": "drunk",
  "name": "Drunk",
  "team": "village",
  "priority": "low",
  "implementation_complexity": "moderate",
  "night_action": true,
  "action_order": 7,
  "action_description": "Must swap their card with a center card but
doesn't look at new role. Doesn't know own role.",
  "win_condition": "Based on final role",
  "special_mechanics": ["forced_swap", "unknown_identity",
  "one_night_only"],
  "reveals_on_death": true,
  "recommended_player_count": "6+ (One Night variant)",
  "quantity_formula": "1"
},
{
  "role_id": "insomniac",
  "name": "Insomniac",
  "team": "village",
  "priority": "low",
  "implementation_complexity": "simple",
  "night_action": true,
  "action_order": 8,
  "action_description": "Last to wake. Looks at own card to see if role
changed during night.",
  "win_condition": "Village team wins",
  "special_mechanics": ["self_check", "detects_swaps",
  "one_night_only"],
  "reveals_on_death": true,
  "recommended_player_count": "6+ (One Night variant)",
  "quantity_formula": "1"
},
{
  "role_id": "alpha_werewolf",
  "name": "Alpha Werewolf",
```

```
"team": "werewolf",
"priority": "low",
"implementation_complexity": "complex",
"night_action": true,
"action_order": 1,
"action_description": "Enhanced werewolf with additional center card
viewing power in One Night variant.",
"win_condition": "Werewolf team wins",
"special_mechanics": ["enhanced_werewolf", "center_card_access"],
"reveals_on_death": true,
"recommended_player_count": "8+ (Daybreak expansion)",
"quantity_formula": "1"
}
]
```

Night Phase Action Order

Traditional Multi-Round Order

```
{
"night_sequence": [
{
"order": 0,
"role": "cupid",
"condition": "first_night_only",
"description": "Creates bond between two players"
},
{
"order": 1,
"role": "werewolf",
"duration_seconds": 30,
"description": "Werewolves see each other and choose victim"
},
{
"order": 2,
"role": "doctor",
"duration_seconds": 15,
"description": "Doctor chooses player to protect"
},

```

```

{
  "order": 3,
  "role": "seer",
  "duration_seconds": 15,
  "description": "Seer investigates one player"
},
{
  "order": 4,
  "role": "witch",
  "duration_seconds": 20,
  "description": "Witch may use potions (if available)"
}
]
}

```

One Night Variant Order

```

{
  "night_sequence_one_night": [
    {
      "order": 0,
      "role": "doppelganger",
      "duration_seconds": 10,
      "description": "Views one player's card and copies that role's action immediately"
    },
    {
      "order": 1,
      "role": "werewolf",
      "duration_seconds": 10,
      "description": "Werewolves see each other. Lone werewolf may view one center card"
    },
    {
      "order": 2,
      "role": "minion",
      "duration_seconds": 10,
      "description": "Sees werewolves (they give signal)"
    },
    {

```

```
"order": 3,  
"role": "mason",  
"duration_seconds": 10,  
"description": "Masons see each other"  
},  
{  
"order": 4,  
"role": "seer",  
"duration_seconds": 10,  
"description": "Views one player's card OR two center cards"  
},  
{  
"order": 5,  
"role": "robber",  
"duration_seconds": 10,  
"description": "Swaps card with player and views new role"  
},  
{  
"order": 6,  
"role": "troublemaker",  
"duration_seconds": 10,  
"description": "Swaps two other players' cards without looking"  
},  
{  
"order": 7,  
"role": "drunk",  
"duration_seconds": 10,  
"description": "Swaps card with center without looking"  
},  
{  
"order": 8,  
"role": "insomniac",  
"duration_seconds": 10,  
"description": "Views own card to check if changed"  
}  
]  
}
```

Game Configuration Rules

Player Count to Role Distribution

```
{
  "distribution_formulas": {
    "werewolves": {
      "7_to_9_players": 2,
      "10_to_12_players": 2,
      "13_to_15_players": 3,
      "16_plus_players": "Math.floor(total_players / 4) with minimum 2"
    },
    "special_roles": {
      "7_players": ["seer"],
      "8_players": ["seer", "doctor"],
      "9_players": ["seer", "doctor", "hunter"],
      "10_plus_players": ["seer", "doctor", "hunter", "additional_roles"]
    },
    "villagers": "total_players - werewolves - special_roles - moderator"
  }
}
```

One Night Card Selection

```
{
  "one_night_setup": {
    "formula": "player_count + 3 center cards",
    "recommended_distributions": [
      {
        "players": 3,
        "cards": [
          "werewolf",
          "werewolf",
          "seer",
          "robber",
          "troublemaker",
          "villager"
        ],
        "total": 6
      },

```

```

{
  "players": 4,
  "cards": [
    "werewolf",
    "werewolf",
    "seer",
    "robber",
    "troublemaker",
    "villager",
    "villager"
  ],
  "total": 7
},
{
  "players": 5,
  "cards": [
    "werewolf",
    "werewolf",
    "seer",
    "robber",
    "troublemaker",
    "villager",
    "villager",
    "villager"
  ],
  "total": 8
}
]
}
}

```

Win Conditions Logic

Traditional Multi-Round

```

function checkWinCondition(gameState) {
  const aliveWerewolves = gameState.players.filter(
    p => p.alive && p.team === "werewolf"
  ).length;

```

```

const aliveVillagers = gameState.players.filter(
  p => p.alive && p.team === "village"
).length;

if (aliveWerewolves === 0) {
  return { winner: "village", reason: "All werewolves eliminated" };
}

if (aliveWerewolves >= aliveVillagers) {
  return { winner: "werewolf", reason: "Werewolves achieved parity" };
}

return { winner: null, gameOngoing: true };
}

```

One Night Variant

```

function checkOneNightWinner(votingResults, finalRoles) {
  const eliminatedPlayers = getPlayersWithMostVotes(votingResults);
  const werewolvesEliminated = eliminatedPlayers.filter(
    p => finalRoles[p].team === "werewolf"
  );

  const werewolvesInGame = Object.values(finalRoles).filter(
    r => r.team === "werewolf"
  ).length;

  // Check Tanner win
  const tannerEliminated = eliminatedPlayers.some(
    p => finalRoles[p].role_id === "tanner"
  );

  if (tannerEliminated) {
    // Tanner wins (others may also win)
    return { winners: ["tanner", ...getOtherWinners()], reason: "Tanner
    eliminated" };
  }

  if (werewolvesInGame === 0) {
    // No werewolves in game
    if (eliminatedPlayers.length === 0) {
      return { winner: "village", reason: "No werewolves exist, no

```



```
elimination" };
}
return { winner: "village", reason: "No werewolves to eliminate" };
}

if (werewolvesEliminated.length > 0) {
return { winner: "village", reason: "At least one werewolf eliminated"
};
}

return { winner: "werewolf", reason: "No werewolves eliminated" };
}
```

Game State Schema

Traditional Game State

```
interface GameState {
game_id: string;
game_mode: "traditional" | "one_night";
phase: "night" | "day" | "ended";
round_number: number;
current_action_role: string | null;

players: Array<{
player_id: string;
username: string;
role: string;
team: "werewolf" | "village" | "neutral";
alive: boolean;
death_round: number | null;
votes_received: number;
}>;

night_actions: Array<{
round: number;
role: string;
action_type: "kill" | "protect" | "investigate" | "other";
target_player_id: string;
```

```

result: any;
}>;

day_votes: Array<{
round: number;
voter_id: string;
target_id: string;
}>;

elimination_history: Array<{
round: number;
player_id: string;
role: string;
eliminated_by: "werewolf" | "village_vote" | "hunter";
}>;
}

```

One Night Game State

```

interface OneNightGameState {
game_id: string;
game_mode: "one_night";
phase: "setup" | "night" | "day" | "voting" | "reveal" | "ended";

players: Array<{
player_id: string;
username: string;
initial_role: string;
current_role: string | null; // null if swapped and unknown
team: "werewolf" | "village" | "neutral";
}>;

center_cards: [string, string, string];

night_actions_completed: Array<{
role: string;
player_id: string;
action: {
type: "view" | "swap";
targets: string[];
viewed_role?: string;
}
}

```

```
};
}>;

discussion_timer: number;

votes: Array<{
voter_id: string;
target_id: string;
}>;
}
```

Strategic Patterns (For AI Players)

Werewolf Team Tactics

```
{
"werewolf_strategies": [
{
"strategy": "target_priority",
"description": "Kill high-value information roles first",
"priority_order": ["seer", "doctor", "hunter", "vocal_villagers"]
},
{
"strategy": "false_claiming",
"description": "Claim power roles to discredit real holders",
"common_claims": ["seer", "doctor", "villager"]
},
{
"strategy": "deflection",
"description": "Redirect suspicion to active villagers",
"indicators": ["accuse_vocal_players", "create_doubt", "split_votes"]
},
{
"strategy": "teammate_defense_timing",
"description": "Only defend werewolf teammate when victory is assured",
"condition": "werewolves >= villagers - 1"
}
]
}
```

Village Team Tactics

```
{
  "village_strategies": [
    {
      "strategy": "information_gathering",
      "description": "Track all role claims and detect contradictions",
      "track": ["claimed_roles", "investigation_results", "protection_claims"]
    },
    {
      "strategy": "behavioral_analysis",
      "description": "Observe deviation from normal speaking patterns",
      "red_flags": ["too_quiet", "too_aggressive", "deflecting",
        "inconsistent_story"]
    },
    {
      "strategy": "role_protection",
      "description": "Keep confirmed village power roles alive",
      "protect": ["confirmed_seer", "doctor", "masons"]
    },
    {
      "strategy": "vote_coordination",
      "description": "Consolidate votes to avoid ties",
      "mechanism": "majority_consensus_before_voting"
    }
  ]
}
```

One Night Deduction Logic

```
{
  "one_night_logic_chains": [
    {
      "scenario": "robber_claim",
      "deduction": "If Robber claims to have swapped with PlayerX and became Villager, then PlayerX should now claim to be Robber",
      "validation": "check_reciprocal_claims"
    },
    {
      "scenario": "troublemaker_claim",
```

```

"deduction": "If Troublemaker swapped PlayerA and PlayerB, their
role claims should be verifiable through other information sources",
"validation": "cross_reference_with_seer"
},
{
"scenario": "lone_werewolf",
"deduction": "If only one werewolf in game, they viewed a center card
and may claim that role",
"counter": "seer_can_verify_center_cards"
},
{
"scenario": "multiple_role_claims",
"deduction": "If two players claim same role (except Mason), at least
one is lying",
"action": "eliminate_one_of_conflicting_claims"
}
]
}

```

Implementation Checklist

Phase 1: MVP (Minimum Viable Product)

```

{
"phase_1_features": [
{
"feature": "room_creation",
"requirements": ["unique_room_code", "shareable_link",
"host_controls"]
},
{
"feature": "role_assignment",
"roles": ["villager", "werewolf", "seer", "doctor"],
"distribution": "automatic_based_on_player_count"
},
{
"feature": "night_phase",
"requirements": ["sequential_role_actions",
"private_communication_werewolves", "action_submission"]
}
]
}

```

```

},
{
  "feature": "day_phase",
  "requirements": ["group_chat", "voting_system", "elimination_reveal"]
},
{
  "feature": "win_detection",
  "requirements": ["check_after_each_elimination",
  "announce_winner", "reveal_all_roles"]
}
]
}

```

Phase 2: Enhanced Features

```

{
  "phase_2_features": [
    {
      "feature": "additional_roles",
      "roles": ["hunter", "tanner", "minion", "mason"]
    },
    {
      "feature": "one_night_mode",
      "requirements": ["center_cards", "role_swapping_logic",
      "simultaneous_voting", "10_minute_timer"]
    },
    {
      "feature": "moderator_automation",
      "requirements": ["automated_announcements", "phase_transitions",
      "timer_management"]
    },
    {
      "feature": "spectator_mode",
      "requirements": ["watch_without_revealing", "see_all_roles",
      "eliminated_player_access"]
    }
  ]
}

```

Phase 3: Advanced Features

```
{
  "phase_3_features": [
    {
      "feature": "custom_configurations",
      "requirements": ["manual_role_selection", "custom_night_order",
        "house_rules"]
    },
    {
      "feature": "statistics_tracking",
      "requirements": ["win_rates_by_role", "player_performance",
        "game_history"]
    },
    {
      "feature": "advanced_roles",
      "roles": ["witch", "cupid", "drunk", "insomniac", "alpha_werewolf"]
    },
    {
      "feature": "voice_chat_integration",
      "requirements": ["webrtc_implementation", "push_to_talk",
        "mute_during_night"]
    }
  ]
}
```

API Endpoint Specifications (Recommended)

Room Management

```
// POST /api/rooms/create
interface CreateRoomRequest {
  host_player_id: string;
  game_mode: "traditional" | "one_night";
  max_players: number;
}
```

```
interface CreateRoomResponse {
room_id: string;
room_code: string;
join_link: string;
}

// POST /api/rooms/{room_id}/join
interface JoinRoomRequest {
player_id: string;
username: string;
}

// POST /api/rooms/{room_id}/configure
interface ConfigureRoomRequest {
roles: string[];
day_phase_duration: number;
allow_no_elimination: boolean;
}

// POST /api/rooms/{room_id}/start
interface StartGameRequest {
host_player_id: string;
}
```

Game Actions

```
// POST /api/games/{game_id}/night-action
interface NightActionRequest {
player_id: string;
action_type: "kill" | "protect" | "investigate" | "swap" | "view";
target_player_ids: string[];
}

interface NightActionResponse {
success: boolean;
result?: {
role_revealed?: string;
is_werewolf?: boolean;
swapped_with?: string;
};
}
```



```
// POST /api/games/{game_id}/vote
interface VoteRequest {
  voter_id: string;
  target_player_id: string;
}
```

```
// GET /api/games/{game_id}/state
interface GameStateResponse {
  phase: string;
  round: number;
  current_action: string | null;
  your_role: string;
  alive_players: number;
  time_remaining: number;
}
```

WebSocket Events

```
// Client → Server
interface WebSocketClientEvents {
  "game:join": { game_id: string; player_id: string };
  "game:action": NightActionRequest;
  "game:vote": VoteRequest;
  "chat:message": { player_id: string; message: string };
}
```

```
// Server → Client
interface WebSocketServerEvents {
  "game:phase_change": { phase: string; duration: number };
  "game:player_joined": { player_id: string; username: string };
  "game:player_eliminated": { player_id: string; role: string };
  "game:night_results": { deaths: string[]; saved: boolean };
  "game:ended": { winner: string; reason: string; all_roles:
Record<string, string> };
  "chat:message": { player_id: string; username: string; message: string
};
}
```

Testing Scenarios

Unit Test Cases

```
{
  "test_scenarios": [
    {
      "test_id": "win_condition_1",
      "scenario": "All werewolves eliminated during day",
      "setup": {
        "alive": ["villager1", "villager2", "villager3"],
        "dead_werewolves": ["werewolf1", "werewolf2"]
      },
      "expected_result": "village_wins"
    },
    {
      "test_id": "win_condition_2",
      "scenario": "Werewolves achieve parity",
      "setup": {
        "alive": ["werewolf1", "werewolf2", "villager1", "villager2"]
      },
      "expected_result": "werewolf_wins"
    },
    {
      "test_id": "doctor_save",
      "scenario": "Doctor protects werewolf target",
      "night_actions": [
        { "role": "werewolf", "target": "player1" },
        { "role": "doctor", "target": "player1" }
      ],
      "expected_result": "player1_survives"
    },
    {
      "test_id": "seer_investigation",
      "scenario": "Seer checks werewolf",
      "night_actions": [
        { "role": "seer", "target": "werewolf1" }
      ],
      "expected_result": { "is_werewolf": true }
    }
  ]
}
```

```

},
{
  "test_id": "hunter_death",
  "scenario": "Hunter eliminated and uses ability",
  "setup": {
    "eliminating": "hunter1"
  },
  "hunter_target": "werewolf1",
  "expected_result": "both_eliminated"
},
{
  "test_id": "one_night_robber_swap",
  "scenario": "Robber swaps with werewolf",
  "night_actions": [
    { "role": "robber", "action": "swap", "target": "werewolf1" }
  ],
  "expected_result": {
    "robber_becomes": "werewolf",
    "werewolf1_becomes": "robber"
  }
},
{
  "test_id": "one_night_no_werewolf_death",
  "scenario": "Village votes wrong player in game with werewolves",
  "setup": {
    "roles": { "p1": "werewolf", "p2": "villager", "p3": "villager" }
  },
  "votes": { "p2": 2, "p1": 1 },
  "expected_result": "werewolf_wins"
}
]
}

```

Common Edge Cases

```

{
  "edge_cases": [
    {
      "case": "tie_vote",

```

```
"traditional_handling": "No elimination, proceed to night",
"one_night_handling": "All tied players eliminated, check win
condition"
},
{
"case": "doctor_self_protection",
"allowed": true,
"limitation": "some_variants_restrict_consecutive_self_heals"
},
{
"case": "hunter_chain_reaction",
"scenario": "Hunter kills another Hunter",
"handling": "Second Hunter also chooses target (chain continues)"
},
{
"case": "lone_werewolf_one_night",
"special_ability": "May view one center card",
"strategic_advantage": "Can claim viewed role"
},
{
"case": "troublemaker_swaps_self",
"allowed": false,
"rule": "Must swap two OTHER players"
},
{
"case": "no_werewolves_in_one_night",
"scenario": "All werewolf cards in center",
"win_condition": "Village wins if no one dies OR any non-villager dies"
},
{
"case": "all_players_dead_simultaneously",
"scenario": "Rare multi-Hunter chain",
"handling": "Game ends, check team with most survivors (or team of
last eliminated)"
},
{
"case": "disconnected_player",
"handling_options": [
"pause_game_temporarily",
```

```
"ai_takeover",
"vote_to_continue_without_player"
]
}
]
}
```

Balancing Guidelines

```
{
  "balance_recommendations": {
    "werewolf_ratio": {
      "7-9_players": "2 werewolves (22-28%)",
      "10-12_players": "2-3 werewolves (20-30%)",
      "13-15_players": "3-4 werewolves (23-30%)",
      "16+_players": "25-30% werewolves"
    },
    "power_role_limits": {
      "seer": "maximum 1",
      "doctor": "maximum 1",
      "hunter": "maximum 1-2 depending on size",
      "villagers": "at least 40% of players"
    },
    "one_night_balance": {
      "werewolves": "2 for 3-8 players, 3 for 9+ players",
      "role_changers": "Maximum 2 (Robber + Troublemaker OR Robber + Drunk)",
      "information_roles": "Seer + at most one other (Minion or Mason)"
    },
    "difficulty_tuning": {
      "favor_village": "Add Doctor, add Masons, add Hunter",
      "favor_werewolf": "Add Minion, add Drunk, reduce information roles",
      "add_chaos": "Add Tanner, add Troublemaker, add Cupid"
    }
  }
}
```

Database Schema Recommendations

Tables

-- Players table

```
CREATE TABLE players (  
  player_id UUID PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Rooms table

```
CREATE TABLE rooms (  
  room_id UUID PRIMARY KEY,  
  room_code VARCHAR(6) UNIQUE NOT NULL,  
  host_player_id UUID REFERENCES players(player_id),  
  game_mode VARCHAR(20) NOT NULL,  
  status VARCHAR(20) DEFAULT 'waiting',  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

-- Games table

```
CREATE TABLE games (  
  game_id UUID PRIMARY KEY,  
  room_id UUID REFERENCES rooms(room_id),  
  phase VARCHAR(20) NOT NULL,  
  round_number INT DEFAULT 1,  
  started_at TIMESTAMP DEFAULT NOW(),  
  ended_at TIMESTAMP  
);
```

-- Game players table

```
CREATE TABLE game_players (  
  game_id UUID REFERENCES games(game_id),  
  player_id UUID REFERENCES players(player_id),  
  role VARCHAR(30) NOT NULL,  
  team VARCHAR(20) NOT NULL,  
  alive BOOLEAN DEFAULT true,  
  death_round INT,
```

```

PRIMARY KEY (game_id, player_id)
);

-- Night actions table
CREATE TABLE night_actions (
  action_id UUID PRIMARY KEY,
  game_id UUID REFERENCES games(game_id),
  round_number INT NOT NULL,
  player_id UUID REFERENCES players(player_id),
  action_type VARCHAR(30) NOT NULL,
  target_player_id UUID REFERENCES players(player_id),
  result JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Votes table
CREATE TABLE votes (
  vote_id UUID PRIMARY KEY,
  game_id UUID REFERENCES games(game_id),
  round_number INT NOT NULL,
  voter_id UUID REFERENCES players(player_id),
  target_id UUID REFERENCES players(player_id),
  created_at TIMESTAMP DEFAULT NOW()
);

```

Performance Considerations

```

{
  "optimization_strategies": {
    "websocket_scaling": {
      "recommendation": "Use Redis pub/sub for multi-server
      deployments",
      "reason": "Synchronize game state across multiple WebSocket
      servers"
    },
    "game_state_storage": {
      "recommendation": "Redis for active games, PostgreSQL for
      completed games",
      "reason": "Fast in-memory access during gameplay, persistent storage
      for history"
    }
  }
}

```

```
},  
"action_validation": {  
  "recommendation": "Server-side validation for all actions",  
  "security": "Never trust client-reported roles or game state"  
},  
"concurrent_actions": {  
  "night_phase": "Allow parallel submissions, process in sequence",  
  "voting": "Collect all votes before processing"  
}  
}  
}
```

Summary for LLM Implementation

When implementing a Werewolf game based on this specification:

1. **Start with essential roles:** Villager, Werewolf, Seer, Doctor (4 roles minimum)
2. **Implement phase system:** Night (sequential role actions) → Day (discussion + voting)
3. **Handle win conditions:** Check after each elimination
4. **Use structured data:** JSON for game state, WebSocket for real-time updates
5. **Validate server-side:** Never trust client for role information
6. **Balance carefully:** 25-30% werewolves, limit power roles
7. **Test edge cases:** Tie votes, simultaneous deaths, role swaps
8. **Scale progressively:** MVP → One Night mode → Advanced roles

This document provides complete machine-readable specifications for implementing both Traditional and One Night Ultimate Werewolf variants.